

ENSE707 • WEEK 7 LABORATORY

# Software Quality Measurement, Metrics and Evidence

---

*Medical Appointment Booking System / Student laboratory guide*

**Metrics are not quality. They are evidence used to understand quality and support decisions.**

**LAB FOCUS** Define useful quality metrics, implement transparent calculations, analyse trends and segments, and audit whether the resulting evidence supports a sound decision.

*This lab extends earlier ENSE707 work without creating another Test Strategy, Test Plan, RTM, generic unit-test suite or full release report.*

## 1. Overview

In earlier labs you created and improved C# systems, reviewed requirements, designed tests, inspected coverage, managed defects and used test-cycle evidence to support a release decision. This lab changes the focus from producing more tests or management documents to deciding what should be measured, how measurements should be calculated, what context they require and how evidence can be collected consistently.

You will continue with the Medical Appointment Booking System. The supplied project contains the existing domain code and MSTest suite, a small metrics-reporting console application, metric and data-loader tests, and synthetic quality data representing several releases.

**CENTRAL QUESTION** What does the evidence tell us, what does it not tell us, and what decision or action should follow?

## 2. Learning outcomes

1. Distinguish a raw measure, a defined metric, a quality indicator and a target.
2. Use a goal-question-metric chain to select measurements that support a decision.
3. Define numerators, denominators, scope, time window, segments and limitations precisely.
4. Calculate and interpret valid-request success rate, P50/P95 latency and segmented results.
5. Use baselines, targets, trends and qualitative evidence to interpret a current value.
6. Recognise misleading aggregation, weak proxy measures, vanity metrics and metric gaming.
7. Audit a metric for clarity, data quality, decision usefulness and possible misuse.

## 3. How this lab extends previous work

| Earlier lab | Already practised                                                             | What Week 7 does differently                                                           |
|-------------|-------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| Week 3      | QA deliverables, governance and review rules                                  | Uses decision context without creating another strategy or plan                        |
| Week 4      | Testable requirements, acceptance criteria and RTM                            | References existing requirement/risk IDs without rewriting the RTM                     |
| Week 5      | Black-box design, data-driven tests, coverage, system and exploratory testing | Uses existing tests and synthetic observations as inputs to measurement                |
| Week 6      | Test-cycle status metrics, triage, control and release recommendation         | Focuses on metric integrity, trends and segmentation rather than another managed cycle |

## 4. Required tools and supplied project

8. .NET 10 SDK and Visual Studio 2022 or VS Code with C# support.
9. GitHub Copilot is recommended but not required.
10. A spreadsheet viewer is optional; all required calculations can be produced by the supplied C# reporter.

### Supplied structure

```
AppointmentBooking/
|-- AppointmentBooking/           Existing domain project
|-- AppointmentBooking.Tests/     Existing MSTest suite
|-- AppointmentBooking.Metrics/   Metrics-reporting console application
|-- AppointmentBooking.Metrics.Tests/ Supplied metric and loader tests
|-- LabData/
|   |-- booking-events.csv
|   |-- release-history.csv
|   |-- quality-targets.json
|   |-- user-feedback.md
|-- StudentWork/
|   |-- MetricDefinitions.md
|   |-- QualityEvidenceNote.md
|-- AppointmentBooking.slnx
```

## 5. Project preparation

Download and extract the supplied Week 7 starter project to a writable folder. Work from the extracted project root so that the relative paths used by the commands and data files remain valid.

11. Extract AppointmentBooking\_Week7\_Starter.zip.
12. Open the extracted folder in Visual Studio 2022 or Visual Studio Code.
13. Restore the solution and run the existing test suite before changing the project.
14. Run the starter metrics reporter once and inspect the generated quality-summary.md file.

```
dotnet restore AppointmentBooking.slnx
dotnet test AppointmentBooking.slnx

dotnet run --project AppointmentBooking.Metrics \
  --events LabData/booking-events.csv \
  --targets LabData/quality-targets.json \
  --output artifacts/quality-summary.md
```

**CHECKPOINT** The Week 6 domain tests and the supplied loader tests should pass. Three metric-calculation tests are intentionally inconclusive/skipped until Activity 3 is implemented. Record the discovered, passed, skipped and failed tests, but do not create another baseline report or add new tests in this activity.

## 6. Working with GitHub Copilot

You are encouraged to use GitHub Copilot while writing the C# metrics code in this lab. Copilot may help explain unfamiliar syntax, draft a method, suggest grouping logic, or diagnose a calculation error. AI output is a proposal, not evidence that the code is correct.

15. Read and understand every generated line before accepting it.
16. Check that formulas match the metric definition, especially the denominator and treatment of missing data.
17. Run the supplied tests and inspect the actual output.
18. Reject invented filenames, commands, data columns, APIs or assumptions about the dataset.
19. Keep one short record containing the prompt, one useful suggestion and one correction or rejection with your reason.

**SUGGESTED PROMPT** Help me implement this method in the supplied C# project. Use only the fields and requirements shown below. Explain the denominator, empty-data behaviour and edge cases before providing code.

## Activity 1 - Understand the measurement context

Open the files in LabData and identify what each file represents. The data is synthetic and contains no real patient information. Do not alter the supplied evidence to make the results appear better.

| File                 | Purpose                                                                                     |
|----------------------|---------------------------------------------------------------------------------------------|
| booking-events.csv   | Per-request observations such as release, validity, outcome, time band, channel and latency |
| release-history.csv  | Comparable values from previous releases for trend interpretation                           |
| quality-targets.json | Targets and warning thresholds used to interpret current observations                       |
| user-feedback.md     | Qualitative observations that may help explain numerical patterns                           |

Answer the following questions in StudentWork/QualityEvidenceNote.md:

20. Which values are raw observations or measures?
21. Which values are targets rather than observations?
22. Is a request rejected because mandatory input is missing a system failure? Explain.
23. Why is the phrase booking success rate incomplete?
24. What important context would be lost if all releases and time bands were combined?

## Activity 2 - Define metrics from quality questions

Complete StudentWork/MetricDefinitions.md. Begin with a quality goal and a question; do not begin by choosing an attractive number for a dashboard. Define the following three metrics.

25. Valid booking success rate.
26. P95 booking latency.
27. Peak-hour system-failure rate.

For each metric, provide all of the following:

28. quality goal and question;
29. metric name and formula;
30. numerator and denominator, where applicable;
31. included and excluded observations;
32. release, environment and time scope;
33. segment or breakdown that should be inspected;

- 34. baseline and target or warning threshold;
- 35. decision or action the metric supports;
- 36. one limitation or possible misuse.

### Example metric contract

| Field       | Example definition                                                |
|-------------|-------------------------------------------------------------------|
| Goal        | Improve the reliability of appointment booking                    |
| Question    | What proportion of valid booking requests completed successfully? |
| Numerator   | Successful valid booking requests                                 |
| Denominator | All valid booking requests in the defined scope                   |
| Excluded    | Requests rejected because mandatory input was invalid             |
| Scope       | Current release using the supplied production-like dataset        |
| Segment     | Normal hours compared with peak hours                             |
| Action      | Investigate when the target is missed or the trend deteriorates   |
| Limitation  | Does not identify the cause or directly measure usability         |

**METRIC INTEGRITY** A percentage without a defined denominator can be misleading. A value with no scope, comparison or decision may be precise but not useful.

## Activity 3 - Extend the C# metrics reporter

Open AppointmentBooking.Metrics. The starter application already loads BookingEvent records from booking-events.csv and prints raw counts. Add the following calculations to MetricsCalculator.cs. Adapt the namespace only if the supplied project uses a different namespace.

### Part A - Calculate valid-request success rate

Only valid requests belong in the denominator for this metric. Invalid-input rejections are still useful observations, but they answer a different quality question.

```
public static double ValidSuccessRate(IEnumerable<BookingEvent> events)
{
    var validAttempts = events
        .Where(e => e.IsValidRequest)
        .ToList();

    if (validAttempts.Count == 0)
    {
        throw new InvalidOperationException(
            "Valid-request success rate cannot be calculated without valid requests.");
    }

    int successes = validAttempts.Count(e =>
        string.Equals(e.Outcome, "Success",
            StringComparison.OrdinalIgnoreCase));

    return Math.Round(successes * 100.0 / validAttempts.Count, 2);
}
```

Explain in your notes:

37. Why would returning 0% when there are no valid requests be misleading?
38. What different question would successful requests divided by all requests answer?

### Part B - Calculate a percentile

Use the nearest-rank definition in this lab. Sort the observations, calculate ceiling(percentile x number of observations), and select that one-based rank. State the method because different tools can use different interpolation conventions.

```

public static double Percentile(
    IEnumerable<double> observations,
    double percentile)
{
    if (percentile <= 0 || percentile > 1)
    {
        throw new ArgumentOutOfRangeException(
            nameof(percentile),
            "Percentile must be greater than 0 and no greater than 1.");
    }

    var sorted = observations.OrderBy(value => value).ToArray();

    if (sorted.Length == 0)
    {
        throw new InvalidOperationException(
            "A percentile cannot be calculated without observations.");
    }

    int rank = (int)Math.Ceiling(percentile * sorted.Length);
    return sorted[rank - 1];
}

```

Use the method to calculate P50 and P95 latency for the current release.

## Part C - Segment valid-request success by time band

```

public static IReadOnlyDictionary<string, double>
    ValidSuccessRateByTimeBand(IEnumerable<BookingEvent> events)
{
    return events
        .Where(e => e.IsValidRequest)
        .GroupBy(e => e.TimeBand, StringComparer.OrdinalIgnoreCase)
        .ToDictionary(
            group => group.Key,
            group => Math.Round(
                group.Count(e => string.Equals(
                    e.Outcome,
                    "Success",
                    StringComparison.OrdinalIgnoreCase))
                    * 100.0 / group.Count(),
                2),
            StringComparer.OrdinalIgnoreCase);
}

```

Run the supplied MetricsCalculator tests, then run the console application against the supplied data. Do not add general booking tests; the target of this activity is the measurement logic.

```

dotnet test AppointmentBooking.slnx

dotnet run --project AppointmentBooking.Metrics \
  -- --events LabData/booking-events.csv \
  --targets LabData/quality-targets.json \
  --output artifacts/quality-summary.md

```

**EXPECTED REPORT CONTENT** Current release, valid-request success rate, target, P50 latency, P95 latency, normal-hour success, peak-hour success and whether each defined target is met.

## Activity 4 - Interpret trends, segments and qualitative evidence

Update StudentWork/QualityEvidenceNote.md using the generated report, release-history.csv, quality-targets.json and user-feedback.md. Select evidence that helps answer an important question; do not combine unrelated values into one overall quality score.

| Signal            | Comparison or context                     | Interpretation                                         | Action and limitation                                      |
|-------------------|-------------------------------------------|--------------------------------------------------------|------------------------------------------------------------|
| Valid success     | Current value, target and prior releases  | Is reliability stable, improving or deteriorating?     | State one justified action and what the value cannot prove |
| P95 latency       | Compare with P50 and the previous release | Is a minority experiencing unusually slow responses?   | Identify a next investigation; do not claim a cause        |
| Time-band segment | Peak versus normal hours                  | Is risk concentrated rather than system-wide?          | Prioritise the affected context and state uncertainty      |
| User feedback     | Compare comments with numerical patterns  | Does qualitative evidence help explain the experience? | State sample and interpretation limits                     |

Your note must answer:

39. What does the trend reveal that the latest snapshot does not?
40. What does segmentation reveal that the overall result conceals?
41. Why could average latency appear acceptable while some users still experience long waits?
42. How does qualitative feedback complement quantitative evidence?
43. Which four signals would you retain on a small decision-oriented dashboard, and why?
44. Identify one vanity metric for this project and explain why it is weak.
45. State one action supported by the evidence, one inference and one important unknown.

**CAUTION** A high aggregate pass or coverage percentage must not override a critical failed behaviour. Thresholds support judgement; they do not replace it.

## Activity 5 - Audit and strengthen the metric design

Choose one metric from the lab and evaluate it using the questions below. Then revise its contract in StudentWork/MetricDefinitions.md so that another person could calculate and interpret it consistently. Complete this activity independently if it is not finished during the supervised session.

46. What exactly is measured?
47. What scope, denominator, data source and assumptions apply?
48. Compared with what baseline, target or previous result?
49. What decision or action does it support?
50. What missing, invalid or duplicated data could change the result?
51. Could people improve the number without improving the software? How?
52. What important quality concern remains outside this metric?
53. Should the metric be communicated as a snapshot, trend, segmented result or threshold comparison? Justify the choice.

Finish by recording one concrete change you made to the metric definition and explain why the revised version is less ambiguous or less likely to be misused.

## Submission and completion evidence

54. Updated AppointmentBooking.Metrics code with the required calculations.
55. StudentWork/MetricDefinitions.md containing three complete metric contracts.
56. StudentWork/QualityEvidenceNote.md containing evidence, interpretation, action and limitations.
57. A generated artifacts/quality-summary.md report containing the expected calculated indicators.
58. A test result showing that the supplied domain, loader and metric-calculation tests complete successfully.
59. One short Copilot record: prompt, useful suggestion, correction or rejection, and rationale.
60. A completed metric audit with one justified improvement to the selected metric contract.

## Quality check before submission

| Check            | Evidence of completion                                                        |
|------------------|-------------------------------------------------------------------------------|
| Metric integrity | Definitions expose denominator, scope, exclusions, comparison and limitations |

| Check                    | Evidence of completion                                                    |
|--------------------------|---------------------------------------------------------------------------|
| Calculation integrity    | Supplied tests pass and the report agrees with the declared formulas      |
| Data integrity           | Missing, invalid and duplicated observations are considered explicitly    |
| Reproducibility          | The documented command regenerates the same report from the supplied data |
| Interpretation integrity | Claims distinguish observation, inference and unknowns                    |
| AI integrity             | Copilot-assisted work was reviewed, tested and corrected where necessary  |

## Concept summary

| Concept      | Meaning in this lab                                                                          |
|--------------|----------------------------------------------------------------------------------------------|
| Measure      | A raw observation such as a request outcome or latency value                                 |
| Metric       | A defined calculation used to answer a quality question                                      |
| Indicator    | An interpretation suggesting a condition, trend or risk                                      |
| Target       | The desired or required comparison value                                                     |
| Baseline     | A reference point such as a previous release                                                 |
| Threshold    | A comparison value that supports attention or action but does not explain a cause            |
| P95 latency  | The value at or below which 95% of observed latencies fall under the stated method           |
| Segmentation | Breaking an aggregate into meaningful groups to expose concentrated problems                 |
| Data quality | The fitness of observations for the metric, including completeness, validity and consistency |

## Official references

61. **Microsoft Learn: dotnet test:** <https://learn.microsoft.com/en-us/dotnet/core/tools/dotnet-test>
62. **Microsoft Learn: Order data using LINQ:** <https://learn.microsoft.com/en-us/dotnet/csharp/linq/standard-query-operators/ordering-data>
63. **Microsoft Learn: Math.Ceiling:** <https://learn.microsoft.com/en-us/dotnet/api/system.math.ceiling>